September 25, 2026

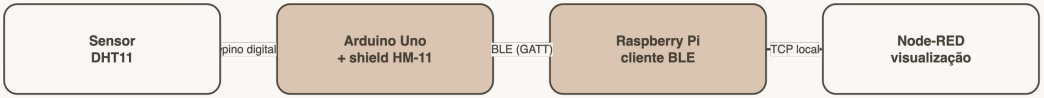
Agenda (1/2)

- Developing BLE applications: architecture of the exercise
- Available libraries and wrappers: BlueZ, bluepy, PyGATT and SimpleBLE
- The bluepy library: class model
- Implementing a scanner
- Working with notifications

Agenda (2/2)

- The BLE server on Arduino: HM-11 shield and serial communication
- Verification on the gateway side

Architecture of the Complete Chain



Architecture of the Exercise (1/2)

- The aim of this lesson is to implement, in practice, the GAP and GATT concepts covered in the previous lesson.
- On the sensor side, an Arduino fitted with a BLE shield takes the peripheral role, that is, the GATT server.
- On the gateway side, a Linux computer (in later lessons, a Raspberry Pi) takes the central role, that is, the GATT client.

Architecture of the Exercise (2/2)

- The client discovers the peripheral, establishes a connection, and reads or writes characteristics to obtain sensor readings.
- This division of roles reproduces the typical architecture of an IoT solution: a constrained node that collects data and a more capable node that aggregates and forwards it.

BLE Libraries and Wrappers (1/2)

- **BlueZ:** the official Linux Bluetooth protocol stack. It supports all core Bluetooth protocols and layers. The `bluez-utils` and `bluez-firmware` packages provide the low-level utilities and the required firmware.
- **bluepy:** a Python interface for BLE, built on top of BlueZ. It is the library used in this course unit.
- **PyGATT:** a Python alternative, with an interface centred on GATT operations.

BLE Libraries and Wrappers (2/2)

- **SimpleBLE:** a library with a simplified API, oriented towards common use cases.
- The choice of library determines the level of abstraction available, but on Linux they all rest on the same BlueZ stack.

bluepy: Class Model (1/2)

- **Peripheral:** represents a peripheral device to which the program connects. It is the entry point for all operations on a given device.
- **Scanner:** searches for BLE devices advertising in the vicinity.
- **ScanEntry:** represents a device found during scanning, with its address and advertising data.
- **DefaultDelegate:** a base class whose methods are invoked when asynchronous events occur, such as the discovery of a device or the arrival of a notification.
- **UUID:** represents service and characteristic identifiers.

bluepy: Class Model (2/2)

- **Service, Characteristic and Descriptor:** they correspond directly to the GATT hierarchy studied in the previous lesson.
- The mapping between these classes and the GATT concepts is direct, which eases the transition from theory to implementation.

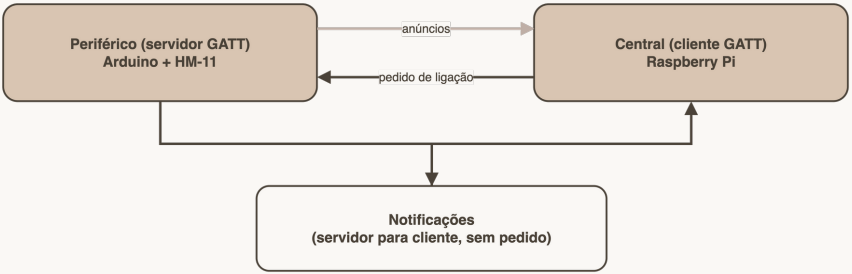
Implementing a Scanner (1/2)

- Scanning is the first step of any client application: without discovering the peripheral, no connection is possible.
- A subclass of `DefaultDelegate` is created and the `handleDiscovery` method is overridden, receiving the device found and two indicators: whether it is a new device and whether it carries new data.
- A `Scanner` is instantiated and the delegate is attached to it through the `withDelegate` method.
- `scan` is invoked with the intended duration, in seconds, returning the list of devices found.

Implementing a Scanner (2/2)

- For each device, the address (`addr`), the address type and the received signal strength (RSSI), in dBm, are available.
- The RSSI value is useful for estimating proximity, although it is not a reliable measure of distance.

Roles and Direction of Exchanges



Notifications: Concept (1/2)

- Reading a characteristic periodically forces the client to query the server repeatedly, even when there is no new data, a simple approach but inefficient in consumption.
- Notifications reverse the initiative: it is the server that informs the client whenever the value of a characteristic changes.
- In the protocol, notifications and indications are distinguished. A notification requires no confirmation from the client; an indication does, which makes it more reliable and more costly.

Notifications: Concept (2/2)

- This mechanism is particularly suited to sensors reporting sporadic events, such as motion detection or a threshold being crossed.
- It is also the mechanism underpinning integration with a message broker, addressed later in the course unit.

GATT Operations and Energy Cost

Leitura (read)	Escrita (write)	Notificação (notify)
O cliente pede, o servidor responde	O cliente envia um valor	O servidor envia sem ser pedido

Sondagem periódica consome energia; a notificação só transmite quando há novidade

Notifications: Implementation in bluepy (1/2)

- A subclass of `DefaultDelegate` is defined and the `handleNotification` method is overridden, receiving two arguments: the *handle* of the characteristic that generated the notification and the data received.
- The *handle* allows the source to be distinguished when several characteristics are configured to notify.
- The `Peripheral` object is created with the device address and the delegate is attached through `setDelegate`.
- The intended service is obtained with `getServiceByUUID` and, from it, the characteristic with `getCharacteristics`.

Notifications: Implementation in bluepy (2/2)

- The notification is enabled by writing to the characteristic's configuration descriptor.
- The program then enters a waiting loop, calling `waitForNotifications`; whenever a notification arrives, the `handleNotification` method is invoked automatically.

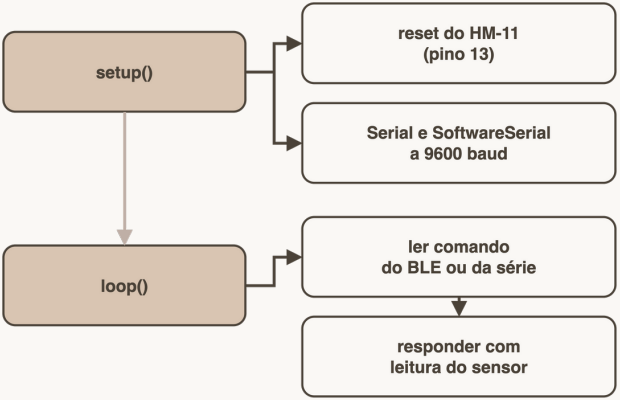
The HM-11 BLE Shield (1/2)

- The practical exercise uses a BLE communications shield fitted on the Arduino, based on the HM-11 module.
- The operating mode is configured by placing *jumpers* on the shield, before mounting it on the board.
- The module communicates with the Arduino over a serial port, which requires a software serial port to be defined, since the physical serial port is taken by the connection to the computer.

The HM-11 BLE Shield (2/2)

- The module's *reset* pin is driven by an Arduino digital output, which allows the module to be reinitialised at program start-up.
- Both serial ports (the physical one, for the serial monitor, and the virtual one, for the BLE module) operate at 9600 bit/s.

Structure of the Arduino Sketch



Structure of the Arduino Program (1/2)

- In the `setup` function, the *reset* pin is configured as an output and the module is reinitialised through a pulse, followed by initialisation of the two serial ports.
- In the `loop` function, the program checks whether data is available from the BLE module or from the serial monitor, and reads the character received.
- The character received acts as a command: `t` for temperature, `h` for humidity, `l` for light level and `X` for the set of all quantities.

Structure of the Arduino Program (2/2)

- Each command has a corresponding response sent simultaneously to the BLE module and to the serial monitor, which eases diagnosis during development.
- Each command also defines a scale factor applied to the loop delay, allowing the response periodicity to be adjusted according to the quantity requested.

Verification on the Gateway Side (1/2)

- The `hcitool dev` command lists the Bluetooth adapters available on the system, with their identifiers and physical addresses.
- The `hcitool lescan` command starts scanning for BLE devices in the vicinity, showing the addresses found and, when available, the name advertised by the device.
- Once the peripheral address is identified, the connection is established with `gatttool -b <address> -I`, which opens the session in interactive mode.
- Within that session, the `connect` command establishes the actual connection to the device.

Verification on the Gateway Side (2/2)

- Discovery of the exposed services and characteristics follows, along with reading or writing their values.
- This command-line verification should precede development in Python: it confirms that the connection is possible before the complexity of code is introduced.

Recommended Working Method (1/2)

- Prepare and test the hardware in isolation: verify the sketch through the serial monitor before involving BLE communication.
- Confirm the visibility of the peripheral with `hcitool lescan`, ensuring that the module is in fact advertising.
- Establish the connection manually with `gatttool` and explore the available services and characteristics.

Recommended Working Method (2/2)

- Only then move on to development in Python, starting with scanning and progressing to characteristic reading and to notifications.
- This progression isolates the source of problems: a failure detected at step three cannot be attributed to the Python code, which has not yet been written.

Lesson Summary (1/2)

- BLE programming translates the GATT hierarchy directly: services, characteristics and descriptors have counterparts in the classes of the libraries used.
- On Linux, the BlueZ stack is the basis on which the higher-level libraries rest, among them the bluepy library used in this course unit.
- The delegation pattern (a subclass of `DefaultDelegate` overriding `handleDiscovery` or `handleNotification`) is the mechanism through which the application reacts to asynchronous events.

Lesson Summary (2/2)

- Notifications replace periodic polling, reducing consumption and latency, and are the natural mechanism for forwarding readings to a message middleware.
- In the following lessons, the client role passes to a Raspberry Pi, which will act as gateway between the BLE network and the IP network.

Sources

Gonçalves, P. *Apresent5 — BLE Dev* and *guia 4 — arduinoBLE*, TCIoT, previous editions. Basis for the bluepy class model, the scanner, notifications and the HM-11 shield exercise.

Bluetooth SIG. *Bluetooth Core Specification*. Definition of the GATT read, write, notify and indicate operations.

bluepy library documentation: `ianharvey.github.io/bluepy-doc/`

Thank you.

Questions and comments